

Coastdown-Based Estimation of Aerodynamic Drag (CdA) and Rolling Resistance (Cr)

Validation_Aero_Test

February 10, 2026

1 Overview

This document describes how we estimate the aerodynamic drag area (CdA) and rolling resistance coefficient (Cr) of a Formula Student vehicle from straight-line coastdown runs already extracted into per-run CSV files in `Runs/`. The method regresses the measured longitudinal deceleration against the square of speed over selected drag intervals, converting the fitted parameters into CdA and Cr.

Scope of this document In addition to the coastdown model itself, this report documents the full project workflow that produces the final coefficient tables: raw signal export and cleanup, run/interval extraction, per-run quality-control outputs, and the final drag/rolling fits.

2 Project Pipeline (Raw $\rightarrow$ Results)

At a high level, the project workflow is:

1. **Raw logging export $\rightarrow$ unified signals CSV.** Export logged channels and resample to a common time base (10 ms).
2. **Cleaning and signal preparation.** Fix missing values, align units, and generate derived signals (including `GPS1_VEL_MAG_ms` and `IMU_ACCEL_LONG_ms2`).
3. **Interval extraction (segmentation).** Automatically propose coastdown intervals, then optionally review/correct them.
4. **Per-run outputs for QC.** Export `Runs/run_XXX.csv` and diagnostic plots.
5. **Coefficient estimation.** Fit per-run coefficients, optionally pair opposite directions, and summarize by configuration.

Segmentation details This report intentionally does *not* reproduce the run-segmentation design (thresholds, state machine, debug logic). Those details are documented separately in `Docs/drag_run_detection.tex`. Here we only reference the scripts and produced artifacts used by the coastdown estimation.

3 Data and Signals

Input Files

- Per-run time series: `Runs/run_XXX.csv` (e.g., `Runs/run_001.csv`)
- Drag intervals: `Runs/drag_intervals.csv` (columns include `run_id`, `t0_s`, `t1_s`)

Signals Used From each run CSV we use:

- **Speed:** GPS1_VEL_MAG_ms [m s^{-1}]
- **Longitudinal acceleration:** IMU_ACCEL_LONG_ms2 [m s^{-2}]
- **Optional filters:** ICU_Apps_pct [%], ICU_Brakpresf_bar, ICU_Brakpresr_bar

4 Longitudinal Acceleration Signal Construction

The coastdown analysis relies on a physically meaningful longitudinal acceleration signal IMU_ACCEL_LONG_ms2 rather than a raw IMU axis. This signal is constructed in `aero_validation/signals.py` as follows:

1. **Bias estimation from standstill.** Using GPS speed (GPS1_VEL_MAG_ms) and wheel speed (WHEEL_SPEED_kmh), we identify samples where the car is effectively stationary ($v < 0.05 \text{ m/s}$). On these segments we take the median of each raw accelerometer axis (IMU_ACCEL_X/Y/Z_ms2) as a stationary bias and subtract it, yielding bias-corrected axes $a_{x,\text{cal}}, a_{y,\text{cal}}, a_{z,\text{cal}}$.
2. **Reference longitudinal acceleration from speed.** From the best available speed signal (GPS or wheel-based), we form a smoothed magnitude $v_s(t)$ and compute a reference longitudinal acceleration $a_{\text{ref}}(t) = dv_s/dt$ by numerical differentiation. This provides an external estimate of true forward acceleration/deceleration.
3. **Straight-line calibration using steering.** To avoid cornering effects, only samples with small steering angle (ICU_SteeringAngle_deg within $\pm 5^\circ$) and sufficient speed ($v_s > 0.5 \text{ m/s}$) are used. On these “straight and moving” segments we assume that a_{ref} is aligned with the vehicle’s longitudinal direction.
4. **Least-squares projection of IMU onto longitudinal axis.** On the valid samples we solve a linear least-squares problem

$$w_x a_{x,\text{cal}}(t) + w_y a_{y,\text{cal}}(t) + w_z a_{z,\text{cal}}(t) \approx a_{\text{ref}}(t), \quad (1)$$

obtaining weights $w = (w_x, w_y, w_z)$ that best align the IMU axes with the speed-derived longitudinal acceleration. If insufficient valid samples are available, a fallback of $w = (1, 0, 0)$ is used (pure X-axis).

5. **Application to the full run.** The longitudinal acceleration used for coastdown is then

$$a_{\text{long}}(t) = w_x a_{x,\text{cal}}(t) + w_y a_{y,\text{cal}}(t) + w_z a_{z,\text{cal}}(t), \quad (2)$$

stored as IMU_ACCEL_LONG_ms2. This combines bias removal, steering-based straight-line selection, and IMU axis alignment into a single robust signal that closely matches dv/dt along the vehicle’s path.

5 Preprocessing and Filtering

For each interval $[t_0, t_1]$ in `drag_intervals.csv` for a given run, we:

1. Slice the run to $t \in [t_0, t_1]$.
2. Enforce coastdown conditions if available: throttle $\leq 1\%$, front/rear brake pressure $\leq 0.1 \text{ bar}$.

3. Require $v \geq 5 \text{ m s}^{-1}$ to reduce low-speed noise.
4. Define positive deceleration as $a_{\text{dec}}(t) = -a_{\text{long}}(t)$.
5. Lightly smooth a_{dec} with a centered rolling median (window 7 samples).

6 End-to-End Procedure

The full workflow from raw signals to configuration summaries is as follows:

1. **Export and clean the dataset (raw $\rightarrow$ cleaned CSV).** The cleaned, analysis-ready file is typically `Data/Aero_Validation_Signals_cleaned.csv`. It is generated by the data-preparation scripts in `Code/Prepare_Data/`.
2. **Interval extraction (segmentation).** `Code/Run_Extraction/export_drag_runs.py` proposes coastdown intervals from the cleaned dataset and writes `Runs/drag_intervals.csv`. If required, this CSV can be reviewed and adjusted before coefficient fitting. (Segmentation logic is documented in `Docs/drag_run_detection.tex`.)
3. **Per-run outputs for QC.** `Code/Run_Extraction/generate_run_outputs.py` uses `Runs/drag_intervals.csv` to export `Runs/run_XXX.csv` and per-run diagnostic plots (velocity/steering/brakes/accelerations) into `Plots/`.
4. **Slice and clean signals.** For every interval, `Code/Validation/calc_drag_coefficient.py` loads the corresponding `Runs/run_XXX.csv` file, enforces the throttle/brake gates listed above, discards samples below the 5 m/s threshold, and constructs (v^2, a_{dec}) pairs after the rolling-median smoothing. This produces evenly spaced, denoised samples while retaining transients such as brake light spikes for diagnostic plots.
5. **Robust regression.** The script fits $a_{\text{dec}} = kv^2 + c$ with iterative 2σ clipping. Each fit reports k , c , R^2 , the number of retained points, and qualitative flags so outlier runs can be revisited quickly.
6. **Convert to $C_dA/C_d/C_r$.** With the chosen mass m and density ρ (defaults in Section 12), the slope/intercept are mapped to C_dA , C_d , and C_r . Optional drag-force columns evaluate F_d at reference speeds for sanity checks.
7. **Pair opposite runs.** Supplying `-pair-opposites` averages consecutive run IDs inside the same configuration to cancel wind/grade biases (Section 8). The script writes both the raw per-run table (`Validation/drag_coefficients.csv`) and the paired table (`Validation/drag_coefficients_paired.csv`).
8. **Summarize by configuration.** `Code/Validation/summarize_drag_configs.py` groups the per-run coefficients by configuration, computing means, dispersions, and wing deltas (tables in Section 13).
9. **Optional pooled fit.** When we want a single regression using every inlier sample per configuration, `Code/Validation/calc_drag_coefficients_by_config.py` stacks all (v^2, a_{dec}) points before fitting. This sacrifices run-level visibility but increases sample size, serving as a cross-check on the averaged results.

7 Dynamics Model

Neglecting grade and wind, the longitudinal dynamics during coastdown can be approximated by

$$m \dot{v} = -\frac{1}{2} \rho C_d A v^2 - C_r m g \Rightarrow a(v) = \dot{v} = -k v^2 - c, \quad (3)$$

with

$$k = \frac{\rho C_d A}{2m}, \quad c = C_r g. \quad (4)$$

We therefore fit

$$a_{\text{dec}}(v) = k v^2 + c, \quad (5)$$

so that k is the slope versus v^2 and c is the intercept.

Parameter Conversion Given mass m and air density ρ , we compute

$$C_d A = \frac{2mk}{\rho}, \quad C_r = \frac{c}{g}. \quad (6)$$

8 Opposite-Direction Pairing for Wind/Grade Cancellation

Each configuration was executed as alternating out-and-back passes—run $(2n-1)$ and run $(2n)$ were driven on the same straight but in opposite directions. We therefore average the fitted parameters of consecutive run IDs within a configuration to cancel steady wind and road grade effects:

$$a_{\text{fwd}}(v) = k v^2 + c + g \sin \theta + a_{\text{wind}}, \quad (7)$$

$$a_{\text{rev}}(v) = k v^2 + c - g \sin \theta - a_{\text{wind}}, \quad (8)$$

so the mean

$$\bar{a}(v) = \frac{1}{2} (a_{\text{fwd}}(v) + a_{\text{rev}}(v)) = k v^2 + c \quad (9)$$

shares the aerodynamic slope while removing the grade/wind intercept bias. The script writes these paired, grade-compensated coefficients to `Validation/drag_coefficients_paired.csv` for downstream analysis.

9 Drag Coefficient and Drag Force

With frontal area A , the drag coefficient is $C_d = \frac{C_d A}{A}$. The aerodynamic drag force is

$$F_d(v) = \frac{1}{2} \rho C_d A v^2 = \frac{1}{2} \rho C_d A v^2. \quad (10)$$

For the test settings $m = 280 \text{ kg}$, $\rho = 1.225 \text{ kg/m}^3$, $A = 1.20 \text{ m}^2$, the overall (all runs) summary is:

- $C_d A$ mean: 0.7176 m^2 ; C_d mean: 0.5980 .
- Mean drag forces: $F_d(10 \text{ m/s}) \approx 44.0 \text{ N}$, $F_d(15 \text{ m/s}) \approx 98.9 \text{ N}$, $F_d(20 \text{ m/s}) \approx 175.8 \text{ N}$.

10 Estimation Procedure

We regress a_{dec} against v^2 using robust linear fitting with simple sigma-clipping:

1. Ordinary least squares to get initial slope/intercept.
2. Compute residuals; iteratively remove points with $|r| > 2\sigma$ (up to 5 iterations).
3. Refit on inliers; report R^2 and the number of points used.

This procedure is implemented in `Code/Validation/calc_drag_coefficient.py`.

11 Run Grouping by Aero Configuration

User-provided mapping of run IDs to configurations:

- Full aero: runs 1–12.
- No rear wing (front only): runs 13–24.
- No wings (no front, no rear): runs 25–36.
- No front wing (rear only): runs 37–46.

Aggregation and wing contributions are computed by `Code/Validation/summarize_drag_configs.py`.

12 Settings Used

Unless noted otherwise:

- Vehicle mass: $m = 280$ kg (including driver)
- Air density: $\rho = 1.225$ kg/m³
- Gravity: $g = 9.80665$ m/s²
- Minimum speed: 5 m/s
- Brake threshold: ≤ 0.1 bar; throttle ≤ 1 % if available
- Smoothing: rolling median window size 7 (centered)

13 Results

Per-configuration sample means, standard deviations (std), variances (var), and run counts (N):

Config	CdA mean	CdA std	CdA var	N	Cr mean	Cr std	Cr var
Full aero	0.8415	0.0910	0.0083	12	0.0296	0.0057	0.0000
No front wing	0.7777	0.0728	0.0053	10	0.0272	0.0039	0.0000
No wings	0.6180	0.1055	0.0111	12	0.0255	0.0042	0.0000
No rear wing	0.6430	0.0400	0.0016	12	0.0272	0.0043	0.0000

Table 1: Per-configuration statistics (sample std/var).

Wing drag contributions relative to the baseline (no wings):

Component	$\Delta C_d A$ [m ²]
Front wing	0.0250
Rear wing	0.1597
Both wings	0.2236

Table 2: Wing drag increments vs. no-wings baseline.

Overall (all runs, per-run fits): $C_d A$ mean = 0.7176, std = 0.1234, var = 0.01522; C_r mean = 0.02739, std = 0.00469, var = 2.20×10^{-5} with $N = 46$.

oprule Config (paired)	CdA mean	CdA std	CdA var	N_{pairs}	Cr mean	Cr std	Cr var
Full aero	0.8415	0.0585	0.0034	6	0.0296	0.0021	0.0000
No front wing	0.7777	0.0457	0.0021	5	0.0272	0.0017	0.0000
No wings	0.6180	0.0574	0.0033	6	0.0255	0.0014	0.0000
No rear wing	0.6430	0.0053	0.0000	6	0.0272	0.0012	0.0000

Table 3: Per-configuration statistics for paired (opposite-direction averaged) runs.

For the paired runs (opposite-direction averaging, Section 8), per-configuration sample statistics are:

Overall (all paired runs): C_dA mean = 0.7176, std = 0.1056, var = 0.01115; C_r mean = 0.02739, std = 0.00214, var = 4.59×10^{-6} with $N = 23$ pairs.

14 Configuration-Level Pooled Fits

Instead of averaging per-run coefficients, we can regress all inlier samples of a configuration at once. `Code/Validation/calc_drag_coefficients_by_config.py` automates this experiment, reusing the same preprocessing logic as the per-run script but stacking every (v^2, a_{dec}) pair before fitting. The resulting coefficients are:

oprule Config	N_{runs}	Samples	CdA [m ²]	Cd	Cr	R^2
Full aero	12	8551.0000	0.8522	0.7102	0.0286	0.6170
No rear wing	12	9539.0000	0.6384	0.5320	0.0269	0.5190
No wings	12	10 747.0000	0.6192	0.5160	0.0250	0.4730
No front wing	10	7915.0000	0.8559	0.7133	0.0244	0.6420

Table 4: Single pooled regression per configuration.

The pooled CdA values closely match the paired-run means, confirming that averaging run-level fits is not introducing bias. However, pooled fits hide problematic runs (a single bad interval only nudges the global slope slightly), so we treat them as a validation cross-check rather than a replacement for the per-run workflow.

15 Discussion and Modeling Choices

Including Drivetrain Friction A viscous drivetrain loss can be incorporated via a linear-in-speed term, fitting

$$a_{\text{dec}}(v) = k_2 v^2 + k_1 v + c, \quad (11)$$

where k_2 maps to C_dA as before, c maps to $C_r g$, and k_1 captures drivetrain friction proportional to speed. Because v and v^2 are correlated over typical coastdown segments, care is needed: we recommend the v^2 -only baseline for aerodynamic estimation, optionally enabling the v -term with centered v and constraints (e.g., $k_2 \geq 0$) when quantifying drivetrain effects.

Why regress a vs v^2 ? In coastdown, aerodynamic drag scales with v^2 , making a linear model in v^2 natural and well-conditioned. The intercept captures speed-independent rolling resistance.

Robustness to Outliers IMU and GPS-derived signals can contain spikes. Sigma-clipped fitting attenuates their influence without requiring a heavy parametric noise model.

Effect of Downforce Aero downforce increases normal load, which can increase rolling resistance. If downforce changes significantly with speed, some of this effect may bleed into the v^2 slope. The configuration comparison (full/no-wings/etc.) partially isolates component contributions.

Unmodeled Disturbances Small grades and wind are not explicitly modeled inside each single run; they contribute to variance. Pairing opposite-direction passes (Section 8) greatly reduces this bias, and if a reliable grade estimate becomes available, the model can further include an explicit $+g \sin(\theta)$ term.

16 Reproducibility

Commands used to generate the results in this document:

```
# (Optional) Export and prepare a cleaned, resampled dataset
# See Code/Prepare_Data/ for the data preparation scripts.

# Export drag intervals (segmentation) from cleaned data
python Code/Run_Extraction/export_drag_runs.py \
  --csv Data/Aero_Validation_Signals_cleaned.csv \
  --out Runs/drag_intervals.csv

# Export per-run CSVs and QC plots
python Code/Run_Extraction/generate_run_outputs.py \
  --csv Data/Aero_Validation_Signals_cleaned.csv \
  --intervals Runs/drag_intervals.csv \
  --outdir Runs

# Compute per-run coefficients (CdA, Cr) for all intervals
python Code/Validation/calc_drag_coefficient.py \
  --mass-kg 280 --rho 1.225 --min-v-ms 5.0 \
  --intervals Runs/drag_intervals.csv \
  --out Validation/drag_coefficients.csv

# Average opposite-direction runs to cancel wind/grade biases
python Code/Validation/calc_drag_coefficient.py \
  --mass-kg 280 --rho 1.225 --min-v-ms 5.0 \
  --intervals Runs/drag_intervals.csv \
  --out Validation/drag_coefficients.csv \
  --pair-opposites --paired-out Validation/drag_coefficients_paired.csv

# Summarize by configuration and compute wing contributions
python Code/Validation/summarize_drag_configs.py \
  --input Validation/drag_coefficients.csv \
  --out Validation/drag_config_summary.csv

# Optional pooled fit per configuration
python Code/Validation/calc_drag_coefficients_by_config.py \
  --mass-kg 280 --rho 1.225 --min-v-ms 5.0 \
  --intervals Runs/drag_intervals.csv \
  --out Validation/drag_config_pooled_fits.csv
```

17 Limitations and Future Work

- Add grade compensation (from GPS altitude or IMU attitude) to reduce intercept bias (current opposite-run averaging removes most of it but does not provide explicit grade estimates).
- Estimate and subtract wind components using dual-pass averaging or anemometer data for added redundancy beyond the existing paired-run cancellation.
- Model speed-dependent rolling due to downforce explicitly to separate CdA and Cr more cleanly.
- Provide per-run plots of a vs v^2 with inliers and regression lines for quality control.

18 Conclusion

The coastdown method provides consistent estimates across configurations, with clear drag increments from the rear and front wings. Variance across runs is within expected bounds given unmodeled wind/grade and sensor noise. The resulting CdA/Cr values can inform setup choices and serve as baselines for future validation sessions.

19 Figures

Example overview and QC plots generated by the pipeline:

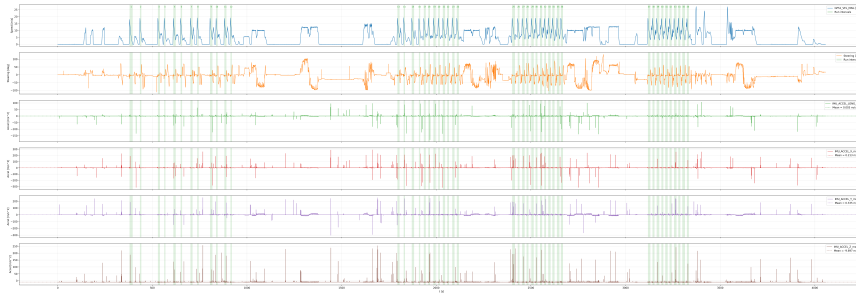


Figure 1: Velocity overview with extracted run intervals (example).

Representative coastdown fits (deceleration vs v^2) with inliers and regression line:

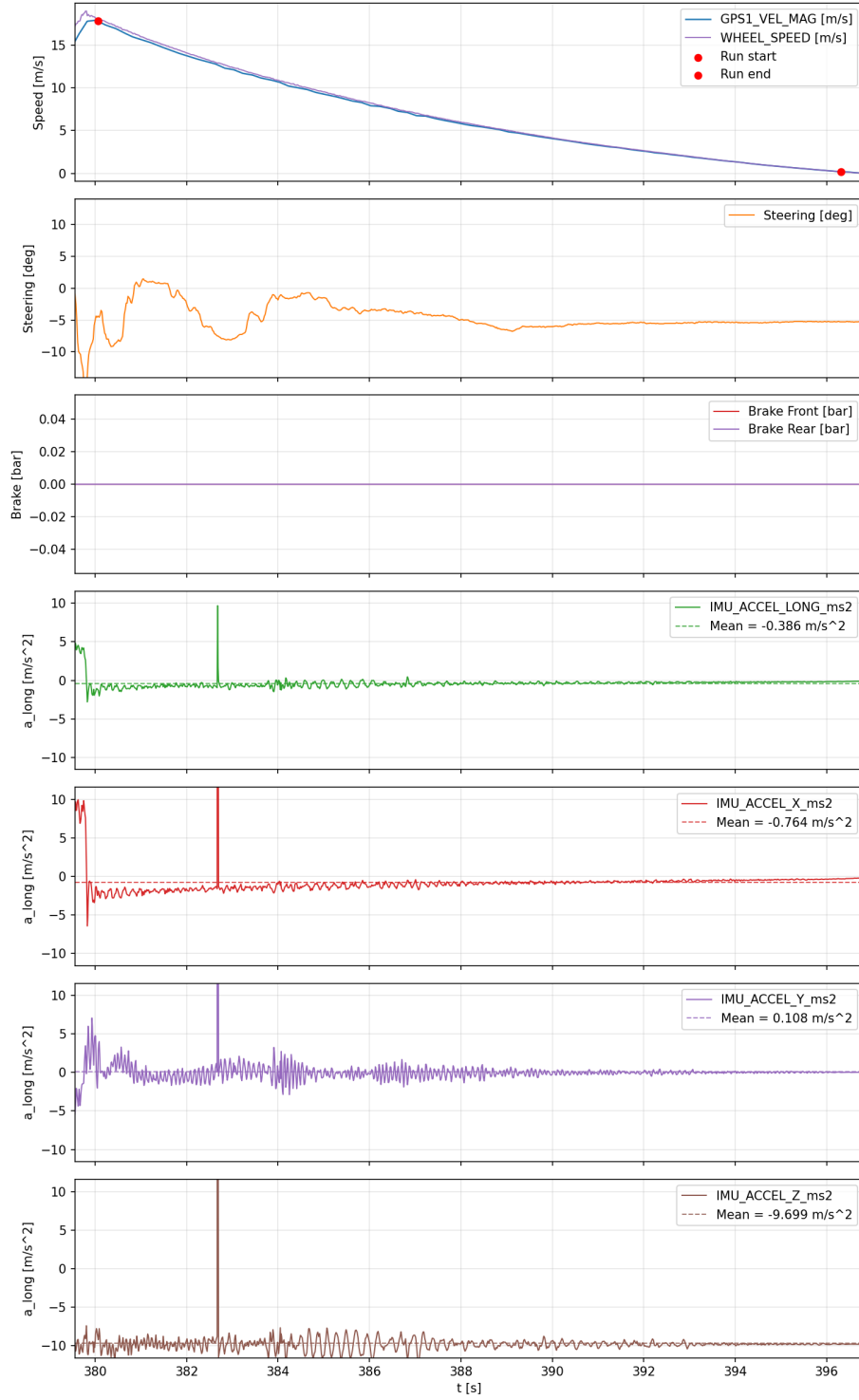


Figure 2: Per-run QC plot (velocity, steering, brakes, and acceleration channels; example run).

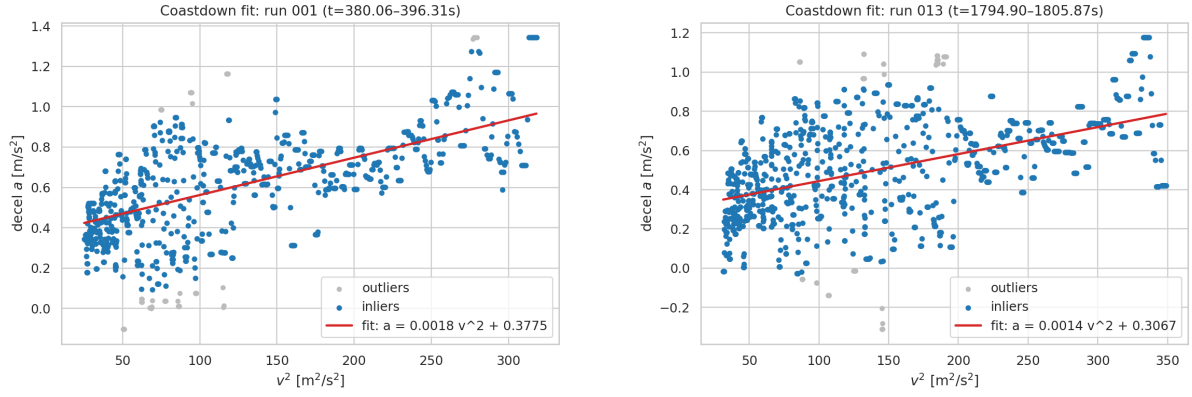


Figure 3: Run 001 (full aero, left) and run 013 (no rear wing, right).

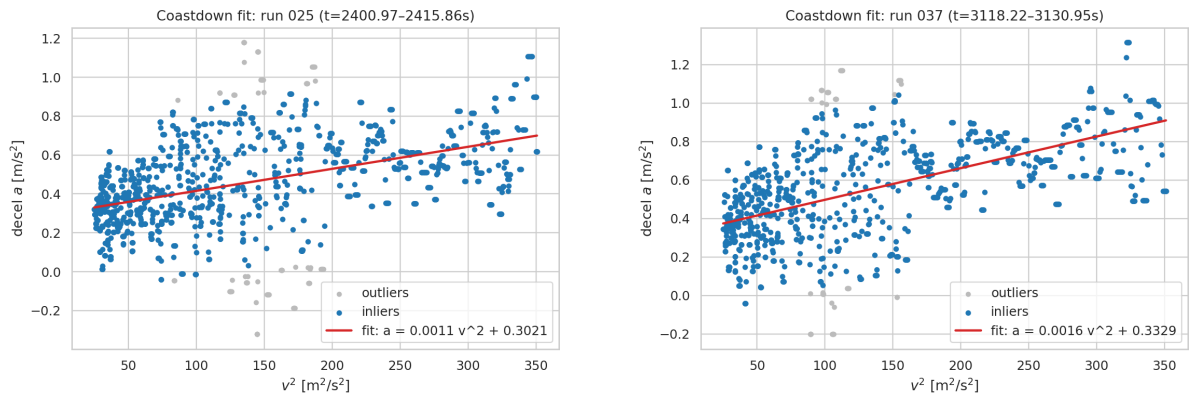


Figure 4: Run 025 (no wings, left) and run 037 (no front wing, right).